Model Training Guide

Train, evaluate, and save every result, step by step

1. Set up Colab

1. Open colab.research.google.com, New notebook, Runtime, Change runtime type, **T4 GPU**.
2. Mount Google Drive and put your dataset and the Starter Code in one folder.
3. Install: `pip install ultralytics scikit-learn` (TensorFlow is already in Colab).
4. Record versions: Python, TensorFlow, ultralytics (run `pip list` and save the output). Needed for reproducibility.

2. Stage 2 first: leaf-health classifier

Folder layout expected: `dataset_split/train|val|test/<class>/*.jpg`. Run three seeds:

```
python train_classifier.py 0
python train_classifier.py 1
python train_classifier.py 2
```

Each run saves `classifier_seedN.json` (accuracy, macro-F1, confusion matrix, per-class report) and the model. Copy the numbers into `Templates/classifier_results.csv`; report mean +/- standard deviation across seeds.

Evaluate on the other test sets (cross-domain and ESP32-CAM)

```
python evaluate_classifier.py classifier_seed0.keras path/to/plantdoc_tomato classes.json
    plantdoc_seed0.json
python evaluate_classifier.py classifier_seed0.keras path/to/own_esp32cam_test classes.json
    own_seed0.json
```

> **WATCH OUT:** PlantDoc and PlantVillage name classes differently. Create a small mapping table (e.g. 'Tomato Early blight leaf' to 'early_blight') and rename the folders so that class names match training exactly. Drop classes that exist in only one dataset and say so.

Also report the binary view

Compute healthy-vs-diseased accuracy and recall by merging all disease classes. For a farm alert, **recall of 'diseased'** matters most (missing a sick plant is worse than a false alarm). Ask an AI tool for the 10-line script (Prompt 4.2) and check its output by hand on a few images.

Ablations that add depth (do these if time allows)

- With vs without augmentation.
- Trained on public data only vs public + your own field images (shows the value of your data).
- Image size 160 vs 224.
- Phone photos vs ESP32-CAM photos in the test set (shows the camera's effect).

3. Stage 1: plant detector

Prepare YOLO-format data: one `.txt` per image with `0 cx cy w h` (normalised). Use a labelling tool such as Roboflow (free tier), CVAT or LabelImg. Negative images get **empty** label files. Then:

```
python train_detector.py
```

It prints precision, recall, mAP50 and mAP50-95 on the test split and exports a TFLite model. Save `runs/plant_v1/` entirely (it contains results.csv, PR curve, confusion matrix and prediction examples: all are figures for the paper).

> **WATCH OUT:** Acceptance test before trusting the detector (from our earlier notes): a person alone must give 0 plants, one whole plant must give 1, a row of plants must give the right count, a pot with soil but no plant must give 0. Record the outcome in a small table.

4. Speed tests

```
python latency_bench.py http://<camera-ip> classifier_seed0.keras
```

This measures 100 camera snapshots and 100 classifier runs and prints median and 95th-percentile times. Note the hardware (CPU model, Colab GPU or laptop) next to the numbers. Do the same for the detector with `model.predict` timing (Prompt 4.3).

5. Error analysis (reviewers love this)

- Open the confusion matrix: which two classes are confused most? Look at 10 examples of each and write one sentence why (similar spots, shadows, motion blur).
- Collect 6-8 failure images (wrong class, missed plant, false plant) for a figure, with their predictions.
- Write down what you would change next. This becomes the Limitations / Future Work text.

6. Save everything

[ ] All training logs and result JSONs
[ ] Confusion matrices and PR curves as PNG (300 dpi)
[ ] The exact class list and split counts per class (a small table)
[ ] Environment versions, random seeds, training time
[ ] A one-paragraph 'what we did' note while it is fresh